

It paged a human at
3 a.m. **to restart a
job.**

Self-healing systems. Why recovery should be a property of the system — not a task for your pager.

[Swipe →](#)

A 30-SECOND TIMEOUT

The alert was almost insulting.

upstream API blips → agent stops
→ marks run failed → pages on-call
→ human wakes, clicks **retry** → succeeds

Human judgment applied: **none**. Human sleep lost: a night. Now multiply that by a fleet.

It could act — but it couldn't **recover.**

Those are not the same capability. We perfect the happy path, then treat every fault as an exception to escalate.

But in production, the **unhappy path is the job.** A system that only works when nothing goes wrong isn't reliable — it's lucky.

The loop.



Notice its own outcome, find the cause, act within bounds, confirm the fix, remember it — without waking anyone.

A remediation is a **hypothesis** — not a fact.

Check the post-condition. Re-run the eval.
Confirm the guardrails still hold — **before** the run continues.

An unverified fix is just a second, more confident failure. **Verify** is what makes self-healing safe instead of scary.

The remediation ladder.

- 0** **Alert only** — detect + suggest a fix
- 1** **Auto-retry the safe stuff** — kills most 3 a.m. pages
- 2** **Propose & approve** — default for state-mutating ops
- 3** **Bounded autonomous** — hard blast radius + verify
- 4** **Autonomous + learning** — feed outcomes to policy

Some failures should **stop** you, not heal.

- **Can't diagnose reliably?** Automation amplifies the guess.
- **Can't verify?** No post-condition = hopeful action.
- **Blast radius unbounded?** A wrong fix beats the failure.
- **Failure is a signal.** Healing has a budget — then fail loud.

Self-healing you can't verify is a liability in a reliability costume.

The self-healing loop — and a remediation-readiness checklist.

Detect it? Diagnose it? Remediation matches the class? Verify every fix? Blast radius bounded? Autonomy matches maturity? Healing has a budget?

Full post + loop diagram + ladder — link in the comments.

Parts 2–7 make it work and get better; this is how it survives. Next: Part 9 — the finale, Securing the Tool Surface.